Similarity Drowns Intent: Three-Trace Sagas and Reinstatement Recall for Provenance in Agentic Software Construction

Ramakrishnan Annaswamy
Independent Researcher
rama@procsolve.com

July 17, 2026

Abstract

Flat similarity retrieval fails *provenance* questions — "why does this code look like this?" — in a characteristic way: it returns restatements and re-askings of the question instead of the conversations where the decision was made. We study these questions over an agent's own recorded history, organized as *sagas* — joint traces of *intent*, *deliberation*, and *artifact* — and present *reinstatement recall*, a seed-conditioned multi-hop retrieval algorithm that is LLM-free and sub-100 ms warm.

Across two within-operator corpora from unrelated codebases, reinstatement improved ground-truth session coverage over one-shot kNN by +53% and +47% at equal budget under pre-registered gates, with zero per-query losses; blind cross-vendor judge panels preferred it on both corpora. On graded provenance gold derived from the operator's own ratification behavior (sealed pre-registered origin recall, dual-vendor dialog-act extraction, external ship-event ledgers), reinstatement wins on origin-MRR (+37%), `nDCG@10` (+29%), and graded recall (+25%) — while both systems miss the verified origin conversation on over half the queries, locating the open headroom. A channel ablation attributes the gain: code-graph spread, not semantic blending, carries the provenance signal. Evaluating the system surfaced a second finding: *self-indexing evaluation contamination*, in which the system ingests the evaluation dialogue itself and echoes of the questions displace the origin conversations being sought. A controlled dose-response experiment from a session-zero snapshot shows five scripted re-ask cycles capturing half of naive retrieval's top-10, with query-echo defenses repairing 72% of the echo occupancy (sham control null). We argue that frozen pre-evaluation snapshots, shared index builds, and echo defenses are necessary controls for any self-recording agent memory. External and multi-operator validation remain open.

Keywords: episodic memory, agent memory, retrieval, provenance, temporal context model, MCP

1 Introduction

In *Programming as Theory Building*, Naur argued that source code is not the primary artifact of software work. The real artifact is the *theory* in programmers' heads — the web of intentions, constraints, and rejected alternatives that explains why the code has the shape it does. When those people leave or forget, the theory is lost, and later maintainers inherit a brittle surface that no longer answers "why" (Naur, 1985).

Agentic software construction sharpens Naur's problem. The "programmer" is no longer a stable human team but a transient human–agent collaboration stretched across many sessions, subagents, and tool traces. Intent is expressed in prompts; deliberation unfolds as reasoning and tool calls (including dead ends and sidechain chatter); artifact materializes as file-level edits. The theory that would justify a later design decision is distributed across those three traces and is *more* perishable than classical human theory-building: sessions end, context windows compact, subagent transcripts fragment into separate files, and the human who held the intent may never re-open the originating conversation.

Most retrieval systems for coding agents treat memory as a flat bag of text: embed utterances, retrieve top-k by cosine similarity, and hope the right chunk appears. For *lexical* or *local* questions ("where is `integrity_check` defined?"), this often works. For *provenance* questions — "why is the integrity check cached?", "why was this API shape chosen?" — flat nearest-neighbor search systematically fails in a predictable way. The query is textually closest to agent restatements, tool-mechanic chatter, or later sessions that *discuss* the decision, not to the human-origin conversation where the decision was made. Similarity drowns intent.

The failure is easy to exhibit on a live system. Asked "why did we drop Qdrant?" on the corpus studied here, one-shot kNN's top hit — at cosine similarity 0.984 — is the transcript of the user asking that same question earlier the same day; ranks two and three are further echoes of the question. The same query under reinstatement recall surfaces the migration analysis, the memory-limit incident preserved in a subagent transcript ("the Qdrant vector database had no memory limits set"), and the months-earlier session in which the replacement storage engine was actually built — a conversation that never contains the words "dropped Qdrant." A retrieval system that answers a why-question with a recording of the question is not a memory; the rest of this paper is about what to build instead, and about how easily measurement itself reproduces this failure mode.

This paper makes two measured contributions and one design contribution, in that order. Measured: (1) *reinstatement recall*, a seed-conditioned multi-hop retrieval algorithm for provenance questions, evaluated against one-shot kNN on two corpora under pre-registered gates and blind cross-vendor judging; (2) *self-indexing evaluation contamination*, a reproducible failure mode of self-recording memory systems, with defenses that measurably repair it. Design: the *three-trace saga* — intent, deliberation, artifact as an organizing structure for agent memory. We are explicit about the boundary: current experiments attribute most of the retrieval gain to the seed-conditioned semantic blend, and source-factored use of the three traces (weighting intent vs. deliberation vs. artifact channels separately) is implemented in storage but not yet ablated; the saga is the architecture

the results motivate, not one they isolate. We implement the system as claude-self-reflect (CSR): a single Rust binary with local MiniLM embeddings (384-dim, FastEmbed), HNSW vector search, SQLite storage, ast-grep code graph construction, fourteen MCP tools, and six Claude Code lifecycle hooks that capture sessions as they happen. The provenance path is reinstatement recall (`csr_why`): seed with one-shot kNN, blend query and seed context for a second hop, spread activation over a session–file code graph, hop one step along episode chains, then fuse and cap.

We evaluate on dogfooded multi-hop questions about CSR's own development history, with pre-registered proceed/iterate/kill gates. The spike passes; production reimplementation is faithful but lower under same-day corpus contamination. That discrepancy is itself a finding: self-indexing memory systems exhibit an observer effect that evaluation design must confront.

2 Related Work

Temporal context and reinstatement. Howard and Kahana's Temporal Context Model (TCM) proposes that episodic recall is driven not only by item–item similarity but by reinstatement of a drifting context representation that bound items together at encoding (Howard & Kahana, 2002). Our context-blend step — forming $0.65 \cdot q + 0.35 \cdot s$ from query vector q and seed vector s, renormalizing, and re-searching — is a deliberate computational analogue: retrieval reinhabits the contextual neighborhood of a retrieved memory rather than matching the literal cue alone. The Context Maintenance and Retrieval (CMR) model extends TCM with source and organizational structure (Polyn, Norman, & Kahana, 2009). CMR motivates treating intent, deliberation, and artifact as distinct but contextually bound sources; our Phase-1 system still fuses them into a single blend rather than source-factored reinstatement, which we flag as future work.

Relevance feedback in information retrieval. Our context-blend step has a clear IR ancestor: Rocchio's relevance feedback (Rocchio, 1971) and its pseudo-relevance variants expand a query toward the centroid of top-ranked documents and re-search — formally close to our $q' = \text{renorm}(0.65q + 0.35s)$. We claim no novelty for vector blending itself. The differences that matter here: PRF re-searches once from a *centroid* of assumed-relevant documents to improve topical recall, while reinstatement walks *per-seed* — each seed spawns its own blended re-query plus graph and episode expansions — and optimizes a different objective (origin-of-decision, not topical relevance). Recent evaluation work sharpens the contrast: Deng (2026), stress-testing agent-memory retrieval under entity collision with pinned BM25 floors and paired-bootstrap intervals, measures lexical PRF (RM3) as a *null* on intent-style memory queries — "no PRF expansion over the lexical channel substitutes for a dense encoder on intent-style queries" — and finds it regresses some strata via query drift. That drift, PRF's canonical failure mode, reappears in self-indexing agent memory in an aggravated form: the corpus continuously ingests restatements of the very queries being asked, so drift is toward the system's own echoes. The query-echo defenses of Phase 1.5 (verbatim-echo demotion, echo-aware seed selection) are, in IR terms, drift countermeasures specialized to self-recording corpora, and we position them as a contribution to that older

literature as much as to agent memory. A dense per-seed centroid-PRF baseline under our shared reranker remains the fair head-to-head and is queued in the ablation grid.

Rational analysis of memory access. Anderson and Schooler's rational analysis argues that human memory's sensitivity to recency and frequency mirrors a Bayesian estimate of future need given environmental statistics (Anderson & Schooler, 1991). This supplies a theoretical warrant for *learned decay*: weighting retrieval by logged access patterns rather than fixed similarity alone. CSR previously logged retrieval only on the prompt-submit hook path; Phase 1 adds MCP-path logging so usage-weighted reinstatement becomes possible once logs accumulate (Phase 2, deferred).

Programming as theory building, and the software-engineering lineage. Naur's thesis frames provenance recall as the product problem: the goal is not merely to find similar text but to reconstitute the theory that produced the code (Naur, 1985). In agentic settings, that theory lives disproportionately in the *intent* trace — human prompts and the human–agent dialogue that fixed constraints — while deliberation and artifact record how and what, respectively. Joint indexing without a path back to intent recreates theory loss at machine speed. The question is empirically grounded in the software-engineering literature: rationale ("why was it done this way?") ranks among the hardest information needs developers report (Ko, DeLine, & Venolia, 2007), and intent/rationale questions dominate developers' hardest-to-answer questions about code (LaToza & Myers, 2010). The classical responses — design-rationale capture from IBIS onward (Kunz & Rittel, 1970; Moran & Carroll, 1996), architecture decision records (Nygard, 2011), and issue-to-commit traceability recovery (Wu et al., 2011) — all require humans to *author or curate* the rationale record. Agentic construction changes the economics: the rationale record now writes itself as a side effect of doing the work, and the open problem moves from capture to retrieval — which is where this paper sits.

Agent memory systems. The established systems closest to this work, in increasing order of relevance. A-MEM (Xu et al., 2025) builds Zettelkasten-style linked memory notes, but its links are constructed at write time and retrieval remains one global cosine kNN — no expansion at query time. Zep/Graphiti (Rasmussen et al., 2025) maintains a bi-temporal entity/fact knowledge graph over conversations and business data with hybrid semantic+BM25+traversal search; it answers "what was true when," over entities rather than code symbols. HippoRAG 2 (Gutiérrez et al., 2025) is the strongest graph-diffusion precedent — seed nodes then Personalized PageRank over an OpenIE triple graph — but diffuses over static QA corpora, not self-recorded agent history. MAGMA (2026) is the nearest neighbor: four orthogonal graphs including LLM-inferred *causal* edges, an intent-aware router that up-weights causal edges for "why" queries, and beam-search traversal. MAGMA's causal graph is entailment over dialogue events (LoCoMo), not AST-anchored code artifacts; its traversal is beam search from rank-fused anchors rather than per-seed reinstatement; and, like every system above, nothing in it guards against the retrieval surface being flooded by echoes of its own recorded queries — the failure mode this paper measures and defends against. Deng (2026) contributes evaluation methodology rather than a system (entity-collision stratification, pinned BM25 floors, bootstrap intervals) and

is discussed under relevance feedback above; we adopt its discipline as the standard our queued ablation grid should meet.

Concurrent 2026 preprints. Three concurrent works occupy immediately adjacent space; the positioning below is based on close reading of the full texts. ACT-Up (Thomson & Lebiere, 2026) is not an agent-memory system but a cognitive-modeling toolkit: a working-memory and spreading-activation module for the ACT-Up architecture, combining base-level decay with spreading activation from working-memory context and validated against human serial-recall data (conditional response probability fits, $r^2 > 0.9$, on the Klein–Addis–Kahana paradigm). Its recall loop is genuinely iterative — "preliminary retrievals continue to be sources of activation, creating a robust chain of contextual information to prime subsequent recall" — which makes it mechanistic *grounding* for our reinstatement walk rather than a competing system: it validates the recall dynamics we borrow, on human data, with no agent, corpus, or code anywhere in scope. MRMS (Li & Shi-Nash, 2026) is architecturally the nearest neighbor: a multi-resolution memory substrate whose typed memory graph carries provenance-relevant edges (*supports, contradicts, supersedes, derived-from*) and whose retrieval pipeline follows kNN with typed graph expansion. It differs on four axes: interaction traces are undifferentiated (no intent/deliberation/artifact taxonomy), there is no code or artifact awareness, its evidence-attribution task is single-hop claim-to-trace selection rather than multi-hop "why" chains, and its evaluation is 800 deterministically generated synthetic tasks with, by its own description, "no private data, production data, external API, or LLM judge" — a complementary regime to the production-corpus evaluation pursued here. E-mem (Wang et al., 2026) preserves *uncompressed* episodic context — raw dialogue/document segments held by per-segment assistant agents — retrieved by multi-pathway activation with an iterative refine-and-query loop, and is the one system of the three with real multi-hop evaluation (LoCoMo, HotpotQA). But its stream is a single undifferentiated token sequence: no artifact or code trace, no human-intent typing, no code graph, and its multi-hop questions are conversational QA, not code provenance. Across all three: none jointly indexes intent+deliberation+artifact, none is code-graph-aware, and none evaluates provenance "why" questions on a production corpus. This paper's contribution is therefore joint three-trace indexing, a specified and measured reinstatement algorithm, and a methodological finding about observer effects in self-indexing systems — not a claim of exclusive novelty in episodic agent memory writ large.

3 System Design

3.1 Architecture overview

CSR is a single Rust binary (`csr-engine`) that replaces an earlier Python/Docker/Qdrant stack. At evaluation time the corpus comprised over 70,000 indexed chunks across roughly 530 conversations, together with stored reflections, an AST-derived code graph, and episode anchors; exact per-snapshot chunk counts appear in the Method section, where they document contamination states. Embeddings are local (MiniLM, 384 dimensions via FastEmbed); search uses HNSW (`hnsw_rs`, less than 1 ms p95 for unconstrained queries); persistence is SQLite (`rusqlite`). AST analysis via ast-grep populates `code_nodes` /

code_edges, linking conversations to files and functions they touched. A code_evolution table records the session↔file timeline used both for graph spreading and for ground-truth construction in evaluation. The system exposes fourteen MCP tools and six Claude Code hooks (SessionStart, UserPromptSubmit, PostToolUse, Stop, PreCompact, SessionEnd) that capture sessions continuously.

3.2 Saga schema (Phase 1 data model)

A *saga* is the joint structure over one unit of work (the term is unrelated to the distributed-transactions Saga pattern; we use it in the narrative sense — the connected story of how a piece of code came to be):

1. **Intent** — human prompts and human-authored dialogue that state goals and constraints.
2. **Deliberation** — agent reasoning, tool-call traces, dead ends, and sidechain/subagent chatter.
3. **Artifact** — resulting code changes, tracked file-by-file and session-by-session in the code graph.

Phase 1 production changes make saga structure explicit in storage:

- **chunks.seq** — import order within a conversation (aligned with the existing chunk index that feeds UUIDv5 chunk ids). Enables future chunk-level temporal reinstatement; not yet used in scoring.
- **chunks.is_sidechain** — true if any source message has the JSONL isSidechain flag *or* the conversation id starts with agent-. Over-labeling was deliberate: live Claude Code places subagent transcripts in separate agent-*.jsonl files, so inline isSidechain flags were effectively absent. The agent- prefix rule labeled 10,907 of 33,015 backfilled chunks. Labeling is present; sidechain-aware rerank weights are deferred.
- **retrieval_events for MCP** — MCP-path searches are logged with hook_phase='mcp_search' and sentinel session_id='mcp'. Previously only the prompt-submit hook path logged retrieval, rendering the dominant MCP recall path invisible to any future frequency/recency mechanism. Logging unblocks Phase 2 learned decay; it does not implement it.

Deferred non-goals (explicitly not claimed as done): learned decay weighting; chunk-level temporal reinstatement via seq; SessionStart saga narration UX; sidechain-aware scoring weights.

3.3 Reinstatement recall (csr_why)

The algorithm implements five steps. Defaults: $k = 10$ (max 50), seeds $= 3$, blend_query_weight=0.65, graph_boost=1.10, graph_cap_per_seed=6, min_score=0.20. No LLM participates in the retrieval path — only vector and graph arithmetic. Warm latency target is less than 100 ms; measured warm latency is 8 ms in production.

Step 1 — Seed retrieval. Compute top-3 nearest neighbors over the query against chunks and stored reflections merged by cosine similarity. This is exactly the current one-shot baseline used by reflect_on_past (minus presentation formatting).

Step 2 — Context blend (second hop). For each seed with vector s, form a blended query

$$q' = \text{renorm}(0.65q + 0.35s)$$

and re-run kNN with q'. This is the TCM-style reinstatement step: search from the context associated with the seed memory, not only from the literal cue.

Step 3 — Code-graph spreading activation. From each seed's session, look up files touched in `code_evolution`, find other sessions that touched the same files, and pull the best-matching chunk (cosine vs. the *original* query) from each neighbor session, with a small activation boost (`graph_boost=1.10`). Cap at `graph_cap_per_seed=6` to limit over-spreading.

Step 4 — Episode chain hop. From the seed session's episode reflection, follow one step of `prev_episode_id` (chronological chain of session summaries) and pull the immediately prior episode when present.

Step 5 — Fuse, rerank, dedupe, cap. Merge channels, deduplicate, apply `min_score`, rerank the fused pool under a provenance-aware policy (Phase 1.5, below), and return at most k results as grouped evidence chains citing conversation ids.

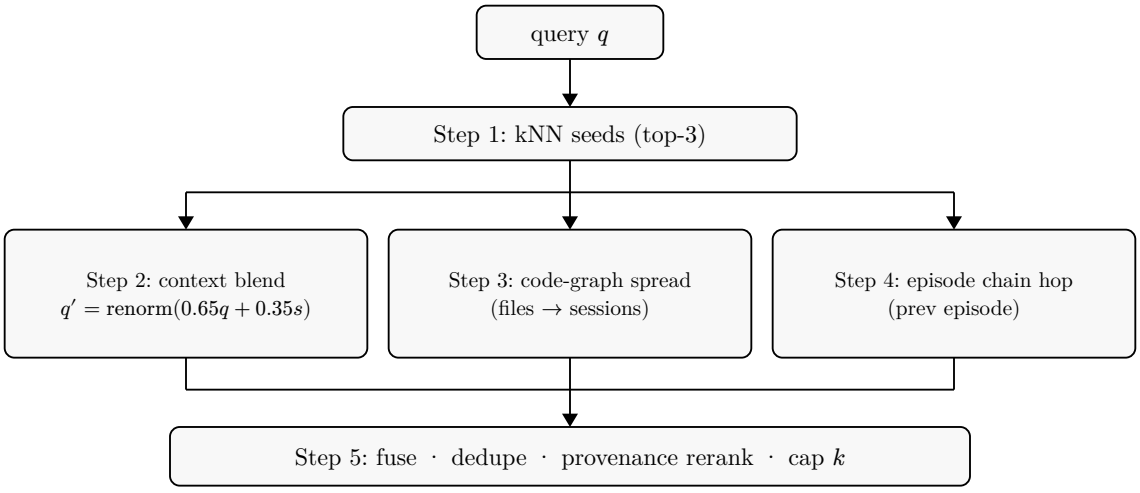

Figure 1: Reinstatement recall. One-shot kNN provides seeds only; three reinstatement channels re-expand from seed context before provenance-aware fusion.

3.4 Provenance-aware rerank (Phase 1.5)

The fused pool is wide (`min_score=0.20`, ~46 candidates in practice), so rank order inside it matters. Phase 1.5 applies CSR's existing rerank machinery to the pool under a dedicated `Provenance` policy, tuned by frozen-corpus evidence rather than by porting `reflect_on_past`'s weights wholesale. Two planned transfers were *rejected* by per-query evidence: demoting tool-mechanic chunks evicted a ground-truth session (edit-heavy chunks are provenance *evidence*, not noise, in this task), and a flat user-authority boost promoted weakly relevant user chunks and user-role compaction summaries over strong evidence. Two defenses were added instead: (1) a **query-echo penalty** (−0.35) on chunks quoting the query near-verbatim — a session that *asks* a question is not the session that *answered* it — paired with echo-aware seed selection (2× seed over-fetch, non-echo hits preferred as walk seeds), and

(2) classifying compaction summaries ("This session is being continued from a previous conversation…") as scaffold text. Without echo-aware seeding, a re-asked question drowns the walk in its own prior askings: in a live test, the reinstatement pool for a repeated query collapsed to three self-referential conversations; with it, five conversations surfaced with the answer-bearing chunk ranked first.

Implementation note on scoring. Per-conversation scoring uses exact cosine over that conversation's stored chunk embeddings (conversations average ~136 chunks — microseconds), *not* a filtered HNSW search. Tiny allowed-id-set filters pathologically escalate `hnsw_rs` toward near-full-index scans; exact scan over a single conversation is both simpler and faster.

3.5 Baseline for comparison

Arm A (baseline) is one-shot kNN over chunks+reflections — the production `reflect_on_past` retrieval core. Arm B is the five-step walk above. Both share the same embedding space and storage. A fairness caveat we state plainly: after Phase 1.5, Arm B also carries the provenance reranker and echo-aware seeding while Arm A does not, so post-1.5 comparisons (including the judge panels) validate the full `csr_why` stack rather than the walk in isolation; only the Phase 0 spike isolates the walk. A factored comparison grid — each arm sharing the identical reranker, dedup, and budget, varying only candidate generation (kNN, kNN+echo demotion, centroid PRF, blend-only, graph-only, full walk) — is specified as the immediate next experiment.

4 Evaluation

4.1 Method

We evaluate *provenance recall*: multi-hop questions of the form "why does the code look like this?", each paired with a ground-truth (GT) target file. GT is the set of sessions in `code_evolution` that historically touched that file. Primary metric **M1** is summed GT-session coverage across queries at equal budget $k = 10$. **M2** is distinct-conversation diversity in the top-10 (to check that gains are not mere result spraying). **M3** is qualitative side-by-side judging (better / tie / worse per query) plus a junk-rate estimate for B-only results; in the spike it was single-annotator, and it was later re-run as a blind three-judge cross-vendor panel with inter-rater agreement reported (see Evaluation §Blind cross-vendor judging).

Pre-registered gates (set before running):

Table 1: Pre-registered gates.

Gate	Criterion
PROCEED	B ≥ A + 25% summed GT coverage **and** B better on ≥50% of queries **and** junk rate under 30%
ITERATE	Mixed results; allow one weight retune and rerun
KILL	B ≤ A; write up honestly, thesis dies cheaply

Disclosed circularity. Arm B's graph-spread step queries the same `code_evolution` table used to define GT, but only via seed sessions found by search: a bad seed yields no graph gain. The design is therefore not circular by construction, but shared plumbing could still inflate graph-channel hits. We therefore require mechanism attribution and M3, not M1 alone.

Attempted independent GT from version control. To break the shared-plumbing dependency entirely, we attempted a second GT construction from VCS metadata: commits in the project repository carry `Claude-Session` trailers naming the web session that produced them, giving a transcript-independent session-to-file mapping. The attempt failed instructively, in two ways. First, coverage: trailers exist on only 60 of 1,096 commits, all within days of writing — months-old decisions targeted by the eval queries predate the convention entirely. Second, and more telling, the bridge from trailer IDs to conversation IDs (locating the trailer text inside indexed transcripts) was itself contaminated by the observer effect this paper documents: the analysis session that ran `git log` to extract trailers thereby imported those trailer IDs into its own transcript, producing spurious mappings to the measuring session. We therefore report no independent-GT agreement number; the honest yield is a protocol recommendation — projects intending to evaluate provenance recall should emit session-to-commit trailers from day one, and should extract them outside the indexed environment.

Corpus. Live CSR development history (dogfooding). Phase 0 spike: morning of 2026-07-15, throwaway read-only binary against 71,762-chunk DB, 12 hand-written queries. Phase 1 production: evening of the same day, `csr-engine eval --provenance`, the same 12-query suite lifted verbatim into the engine, after a full day of development sessions had been imported. The snapshot states used throughout:

Table 2: Evaluation snapshot states.

Snapshot	Chunks	Queries	Contamination state	Index build
Live, morning (spike)	71,762	12 (primary)	pre-eval-dialogue (cleanest)	fresh per run
Live, evening (production)	~73,300	12 (primary)	same-day eval dialogue imported	fresh per run
Frozen `eval-frozen-2026-07-15`	73,342	12 + 8 (second corpus)	partial (includes spike-discussion session)	one shared build per A/B comparison

Statistical treatment. With 12 and 8 queries, we report per-query outcomes and exact paired sign tests on wins/losses (ties excluded) rather than interval estimates: primary suite 7 wins / 0 losses (two-sided $p \approx 0.016$); second corpus 4/0 ($p = 0.125$, not individually significant); pooled 11/0 ($p \approx 0.001$). Summed coverage weights queries with larger GT sets more heavily; per-query win/tie/loss counts are the macro-level check. Bootstrap confidence intervals, macro-averaged normalized recall, and graded origin-level metrics (origin-MRR,

nDCG) require the graded relabeling described under limitations and are queued with the ablation grid.

4.2 Phase 0 spike results

Gate: PASSED.

Table 3: Phase 0 spike results.

Metric	Arm A (kNN)	Arm B (reinstatement)
Summed GT coverage ($k = 10$)	15	23
Relative lift	—	**+53%** (gate +25%)
Per-query outcomes	—	B better 7/12, tie 5/12, worse 0/12
Diversity (distinct conv. in top-10, summed)	75	77

Mechanism attribution of B's GT-hit result lines:

Table 4: Mechanism attribution.

Channel	GT hits
Context blend	24
Graph spread	9
Episode chain	0

The majority of the win is from the pure-semantics blend hop, not the potentially shared-plumbing graph walk — the key quantitative evidence against circularity as the sole explanation of M1.

Qualitative highlight (Q3: "why is the integrity check cached"). Arm A's entire top-10 was subagent/tool-mechanic chunks with **zero** GT sessions represented. Arm B's blend hop surfaced the human-origin conversation (`7eccb720`) where caching was decided and discussed with the user — the failure mode the thesis predicts: one-shot similarity drowns in deliberation chatter while reinstatement digs back toward intent. B-only junk rate was eyeballed under 20%, mostly looser-but-relevant graph pulls rather than noise.

4.3 Phase 1 production results

The same walk was reimplemented as engine code and registered as the 14th MCP tool (`csr_why`). Definition-of-Done checks passed: migrations idempotent; backfill processed 732 candidate files (182 missing/skipped, reported not silently dropped); 8 ms warm latency against a 100 ms budget; well-formed evidence chains.

Table 5: Production runs.

Run	A	B	Notes
Production `eval --provenance` (12 queries)	15	18	First production path
Immediate rerun via original spike code path	15	19	Faithfulness check

Production reimplementation is faithful to the spike algorithm (18–19 vs. the spike's 23 on the same 12-query suite under a dirtier corpus), not a regression to a weaker walk.

4.4 Using the discrepancy as evidence

Spike B=23 (clean, pre-development-session) versus production B=18/19 (post same-day import) is not treated as measurement noise to hide. It is primary evidence for the observer-effect finding developed in Discussion: between morning and evening, transcripts of the development session that *discussed and quoted the eval queries and spike output* were imported by always-on hooks. Self-referential chunks textually outrank true origin conversations for the queries that discuss them. Concretely, the acceptance query about integrity-check caching no longer directly surfaces `7eccb720` as it did in the morning spike.

A frozen snapshot (`eval-frozen-2026-07-15.db`, 4.2 GB, 73,342 chunks) was taken, but even that snapshot includes same-day contamination from the spike-discussion session. Morning pre-development numbers remain the cleaner baseline; future protocol should freeze from a session-zero backup before any evaluation dialogue begins. The necessity of freezing was confirmed directly: across four otherwise-identical live-corpus eval runs during Phase 1.5 development, the chunk count drifted from 73,846 to 73,882 as the session's own transcripts imported, and per-query numbers changed between invocations.

4.5 Phase 1.5 frozen-corpus results

Both binaries (pre- and post-rerank) were evaluated against clones of the frozen snapshot. Aggregate coverage held — A=15, B=18 before and after — while the distribution moved where the thesis predicts: Q3, the acceptance-failure query ("why is the integrity check cached") whose ground-truth session had been displaced by contamination, recovered from 0 to 1 GT sessions; Q12 improved 1 → 2; Q5 and Q10 each ceded one hit (2 → 1, 4 → 3). The rerank is therefore best read as a *contamination repair* mechanism, not a coverage amplifier: it restores origin conversations on the queries where self-echo displaced them, at the cost of marginal hits elsewhere.

Per-query results for the primary suite on the frozen snapshot (a later independent index build of the same corpus, illustrating both the distribution and the documented ±1 ANN build variance — A=14, B=17 on this build vs. 15/18 above):

Table 6: Per-query GT-session coverage, primary suite, frozen snapshot (independent index build).

Query	GT size	A	B	Query	GT size	A	B
Q1	6	2	2	Q7	3	1	**3**
Q2	2	2	2	Q8	0	0	0
Q3	6	0	**1**	Q9	0	0	0
Q4	0	0	0	Q10	6	3	3
Q5	3	2	1	Q11	1	1	1
Q6	3	2	**3**	Q12	2	1	1

Three queries (Q4, Q8, Q9) have zero reachable GT on the frozen snapshot (their `code_evolution` rows post-date it or the target has no file mapping), and B's one per-query loss here (Q5) is the echo-defense trade documented above. The distribution is the honest shape of the result: B's advantage concentrates in a minority of queries where multi-hop reach matters (Q3, Q6, Q7), with the rest tied — consistent with a mechanism that rescues hard cases rather than uniformly inflating scores.

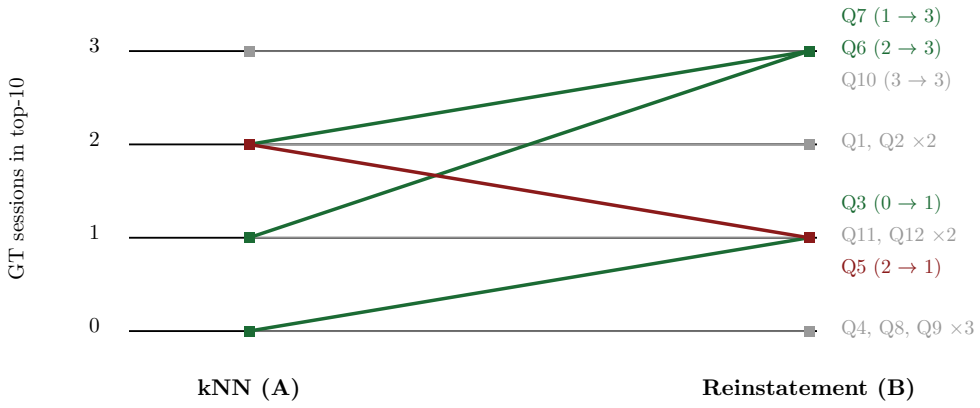

Figure 2: Per-query ground-truth coverage, one-shot kNN (A) vs. reinstatement (B), 12 primary-corpus queries on the frozen snapshot with one shared index build. Rising lines are gains (Q3, Q6, Q7), the falling line is the single regression (Q5, one GT session displaced within budget), flat lines are ties; coincident lines are annotated with their multiplicity. Aggregate: A=14, B=17.

A known remaining gap: for decisions predating the current corpus era (e.g., a major stack replacement), asker sessions still win the top group over the true origin conversation. Chunk-level echo demotion is insufficient when *entire conversations* consist of re-askings; conversation-level echo exclusion and population of a `supersedes` relation are the queued responses.

4.6 Second corpus: within-operator cross-project replication

The corpus is not CSR-only: the same engine continuously indexes every Claude Code project on the machine — 15 projects, 562 conversations, ~74,700 chunks at the time

of writing — of which CSR's own development history is a minority share. The largest corpus family is an unrelated product line (a consumer mobile app, its marketing-campaign operations, and an analytics command center: TypeScript/React/Expo, Meta and PostHog APIs, video production), contributing 3.4× more code-graph rows than CSR itself (613 vs. 180 `code_evolution` rows; 78 vs. 20 sessions; 229 vs. 41 files).

We wrote 8 new provenance queries against this second corpus — authentication-migration decisions, caching-architecture choices, analytics discrepancies, video-composition structure — using the same GT construction (sessions in `code_evolution` touching the target file) and the same frozen snapshot, probed through the production MCP interface with cross-project scope. Coverage: **A=15, B=22 (+47%)**, with B better on 4 queries, tied on 4 (one query missed GT on both arms), and worse on none. The lift on a TypeScript/marketing-operations corpus (+47%) closely replicates the lift on the Rust/systems corpus (+53%), evidence that reinstatement recall's advantage is not an artifact of CSR describing itself.

The blind three-judge cross-vendor panel (same protocol as below) was run on the second corpus as well: reinstatement judged better on 5 of 8 queries, tied on 3, **and worse on none — no judge cast a single vote for the baseline on any query**, with 4 of 8 verdicts unanimous. Fleiss κ is low here (0.19) for a well-understood reason: with zero baseline votes, the only disagreement dimension left is whether a reinstatement win counts as a tie, and κ collapses under such extreme category prevalence (observed agreement 0.67). Judge rationales again converged on origin-surfacing: the baseline drifted into a neighboring project on the video-composition query while reinstatement stayed on the correct project and surfaced the sync-bug root cause.

Two disclosures: query authors knew the GT target files (as in the primary suite), and the corpus — while a different codebase, language, and work domain — is still one operator's history, so multi-operator generalization remains open.

4.7 Blind cross-vendor judging (M3, multi-rater)

To replace the spike's single-annotator M3, we re-ran side-by-side judging with three independent LLM judges from three model families (Claude Sonnet, Grok 4.5, Gemini 3.1 Pro), each invoked headlessly with an identical packet. Evidence was generated fresh for the purpose: a clone of the frozen snapshot served by a single engine process (one index build, so both arms share it), probed over the production MCP interface — Arm A via `csr_reflect_on_past`, Arm B via `csr_why`, both at $k = 10$, all 12 queries. Judges saw the two outputs per query as anonymized sides X and Y with side order randomized per query, were told to judge content rather than formatting, and were asked which side better surfaces the *origin* of the decision, penalizing re-askings and eval-harness echoes.

Majority verdicts: Arm B better on 8 of 12 queries, Arm A better on 1 (Q9, hooks catch-all policy, where the one-shot arm ranked the origin policy statement higher), ties on 3, no three-way splits. Seven of the twelve verdicts were unanimous across all three judges, in both side orders. Inter-rater agreement was moderate: Fleiss $\kappa = 0.51$ (observed agreement 0.78 against 0.55 expected by chance). Judge rationales independently converged on the paper's central mechanism — the modal comment against Arm A was that its top ranks were occupied by the evaluation queries themselves (self-echo), while Arm B surfaced

implementing sessions and root-cause dialogue. Caveats: the judges are LLMs, not humans; the judging instructions were written by the system's authors and explicitly instruct judges to penalize re-askings and evaluation echoes — so the panel is blinded to *side* but not to the paper's preferred failure model (a second panel under a neutral "which side better answers why" prompt, plus a small human-maintainer panel, is the queued remedy); and the packet truncated each side to its first ~4,500 characters.

5 Phase 2 experiments: graded gold, channel ablation, controlled contamination

The three experiments queued in earlier drafts as the next revision's core have now been executed. Together they (i) replace the file-touch GT proxy with graded provenance gold, (ii) attribute the coverage effect to its mechanism, and (iii) convert self-indexing evaluation contamination from a documented incident into a dose-response phenomenon with a quantified repair.

5.1 Graded provenance gold from ratification behavior (E2)

Gold construction. Rather than retrospective annotation, grades were derived from the operator's own contemporaneous dialog-acts and ratification behavior, under a protocol frozen before any retrieval ranks were seen. The operator first answered a sealed, pure-memory elicitation for all 20 queries — committed to version control *before* any cueing or rank exposure — yielding strata: 12 origin descriptions recallable from memory, 7 unresolved (excluded from origin-ranked metrics, never soft-matched), and 1 out-of-corpus. Descriptions were then mapped to conversation IDs by metadata only (literal text search over pre-freeze chunks, git dates, external ship events — no embedding retrieval, which would reintroduce circularity). Candidate pools (both arms' frozen top-10s ∪ date-filtered file-touch history; 211 items, 205 reconstructible) were labeled by two independent LLM extractors from different vendors (Grok 4.5 and Claude Sonnet) applying a quote-anchored extractive protocol for DIRECTS / ACCEPTS / REJECTS / RE-ASKS acts; grading used strict two-vendor consensus, conservative on splits (directs agreement 84.8%, Cohen $\kappa = 0.41$; splits routed to owner audit). Grade 3 was reserved for sealed-and-mapped origins — extraction can corroborate but never mint an origin. External ratification ledgers (git history across four repositories, npm publish timestamps, a release-train manifest) provide off-corpus acceptance evidence: across 204 extracted conversation-items the operator's dialogue contained 21 DIRECTS but only **1 explicit ACCEPTS** — operators direct in words and ratify by shipping, so independently timestamped ship events are the only acceptance signal that exists at scale.

Results (12 mapped-origin queries). Reinstatement beats kNN on every graded metric: origin-MRR 0.264 vs. 0.193 (+37%), `nDCG@10` 0.470 vs. 0.363 (+29%), Recall@10 of grade-$\geq$2 items 0.553 vs. 0.444 (+25%) — direction consistent with the coverage results, now on decision-graded gold. The sobering companion finding: **5 of 12 mapped origins were retrieved by neither arm**, and origin-MRR is zero for both arms on 7 of 12 queries. An owner audit confirmed all five contested mappings correct (map-error reading eliminated); four of the five missed origins date from the corpus's earliest era — old, sparse origin

conversations losing to later, denser sessions. Origin-finding is improved by reinstatement but not solved; the headroom above the winning system is large.

5.2 Channel ablation (E1)

A seven-arm ablation (research harness mirroring the production walk; one process, one shared index build over 73,342 chunks; scored against the E2 gold; conversations outside the graded pool scored zero, a bias *against* exploratory channels) decomposes the walk:

Table 7: Channel ablation on graded gold, one shared index build. Within-grid comparisons only; absolute values are not comparable to the E2 table (different index build).

Arm	**origin-MRR**	**nDCG@10**	**R≥2@10**
kNN baseline	0.243	0.394	0.469
full reinstatement	0.276	0.477	0.559
blend-only	0.269	0.446	0.499
graph-only	**0.329**	**0.555**	**0.607**
episode-only	0.321	0.457	0.470
full − rerank	0.257	0.447	0.563
full − echo-defense	0.234	0.434	0.549

Four findings. (1) *Graph spread is the workhorse*: graph-only beats the full walk on every metric despite the scoring bias against it — code-graph structure, not semantic blending, carries the provenance signal on graded gold. This refines the spike-era attribution: blend dominates when GT is file-touch *coverage*; graph dominates when gold is decision-graded. (2) *Fusion dilutes*: max-score fusion lets blend-sourced semantic neighbors crowd graph-sourced provenance neighbors out of the final budget; the tuning direction is a larger graph share, not more channels. (3) *Echo defense is causally necessary*: removing it drops origin-MRR to 0.234 — below the kNN baseline. (4) *The episode chain, previously a null result, is a strong origin-finder on graded gold* (0.321 origin-MRR) though weak on evidence depth — it finds the origin thread, not the full evidence set.

5.3 Controlled contamination (E3)

From a session-zero snapshot C0 (17,134 chunks, pre-dating all evaluation dialogue) we built C1 = C0 + only the evaluation-design transcript, C-sham = C0 + an unrelated transcript matched for size (107% of C1′s bytes, zero query-text occurrences), and C5 = C0 + five scripted re-ask cycles (verbatim query re-askings with paraphrased answers, explicitly marked synthetic — a controlled self-referential injection, not a natural-ecology observation). Retrieval used exact brute-force scan, eliminating ANN variance so the independent variable is corpus content alone; 8 queries whose owner-audited origins pre-date C0; three arms.

Table 8: Contamination dose-response and repair, exact-scan retrieval, mean over 8 eligible queries.

Arm under C5 (dosed re-asking)	**echo@10**	**origin-MRR**
kNN	4.9 / 10	0.014
walk − echo-defense	4.9 / 10	0.014
walk with echo-defense	**1.4 / 10**	**0.047**

Five scripted re-ask cycles capture half of naive retrieval's top-10 (on one query the entire top-3 becomes the synthetic cycles); the undefended walk is equally captured, since hop-2 spreads outward from echo seeds. The echo defense is a quantified repair: −72% echo occupancy, 3.4× the origin-MRR under dose, and on one query the origin *enters* the top-10 only in the defended arm — demoting echoes clears ranked space for provenance. The sham control is null (rankings byte-identical to C0), so displacement is content-specific, not a corpus-size artifact; and C1 shows the evaluation-design conversation itself entering a query's top-3 — storing a single evaluation session measurably perturbs the system under evaluation. Disclosure: under exact scan on the session-zero corpus, origins sit outside the top-10 for 6–7 of 8 queries in *all* conditions (the origin-finding floor from E2 again); the dose-response and repair rows are the load-bearing results, origin-rank trajectories are floor-limited.

6 Discussion

6.1 What the results support

On this corpus and task, reinstatement recall materially outperforms one-shot kNN for provenance coverage at equal budget. The +53% spike lift cleared a pre-registered +25% gate with zero per-query losses, equal diversity (M2), and qualitative origin-rescue on the canonical failure case (M3). Mechanism attribution places most GT hits in the TCM-style blend channel, supporting the claim that *context reinstatement*, not merely graph co-occurrence, drives the effect. Production DoD confirms the walk can ship as a sub-100 ms, LLM-free MCP tool with honest offline eval.

This is a machine-assisted answer to Naur's theory-loss problem for agentic coding: when theory lives in the intent trace of a saga, retrieval that only matches surface similarity loses the theory; retrieval that reinhabits seed context and spreads through artifact links can recover it (Naur, 1985; Howard & Kahana, 2002).

6.2 Observer effect / corpus self-contamination

We name the phenomenon *self-indexing evaluation contamination* (retaining "Heisenberg-like observer effect" as the informal gloss). The most important methodological contribution is not the coverage table but the observation that **measuring a self-indexing memory system changes the system**. CSR's hooks import the evaluation session while the evaluation is designed and discussed. Near-verbatim restatements of queries and spike dumps become high-similarity competitors to origin conversations — exactly the failure mode one-shot retrieval already suffers, now amplified by the experimenters themselves.

Two consequences follow:

1. **Method.** Any eval of self-recording agent memory must run against a **frozen corpus snapshot** taken *before* the evaluation session can be captured and re-imported. Same-day "freeze after discussion" is insufficient. Prefer a session-zero backup. Report both clean and contaminated numbers when contamination is unavoidable; use the gap as a diagnostic, not as a reason to discard the clean run. A second, smaller comparability requirement surfaced during regression testing: ANN index construction is not deterministic across rebuilds (HNSW insertion effects), and per-query coverage counts near score boundaries can shift by ±1 between two indexes built from the identical frozen corpus (observed: B=18 vs B=17, with two binaries producing per-query-identical results on the *same* index build). A/B comparisons must therefore share one index build, not just one corpus.
2. **Design.** Phase 1.5 applied provenance-aware reranking to the reinstatement pool inside `csr_why` and measured it on the frozen corpus (see Evaluation). The hypothesis — that echo demotion specifically counters contamination — was partially confirmed: query-echo defenses restored origin-surfacing on the canonical displaced query (Q3, $0 \rightarrow 1$) while holding aggregate coverage, but chunk-level demotion does not rescue decisions whose re-askings dominate whole conversations. Notably, two rerank heuristics that work in general-purpose search (`reflect_on_past`) were *rejected* by evidence in the provenance task: tool-mechanic chunks are evidence there, not noise, and flat user-authority boosts promote weak user chatter over strong evidence. Rerank policies are task-relative.

Self-indexing evaluation contamination is a class-level issue, not a CSR-only bug: any agent that indexes its own tool use while researchers discuss eval items will inject high-similarity confounds.

Three live sightings during the preparation of this paper. The effect was not confined to the benchmark. (1) The attempted VCS-trailer GT construction was contaminated by its own extraction step (see Method): running `git log` inside an indexed session imported the trailer IDs being extracted. (2) A file-history lookup made while drafting this paper returned, as its top hit, the session drafting this paper. (3) Most instructively, CSR's own context-injection hook exhibited the failure in its selection layer: asked whether session-start continuity had captured the current work, the hook's semantic intent classifier matched the prompt to a six-day-old episode consisting of *asking about that same hook* — at similarity 0.55, exactly its acceptance threshold — and attached its resume-context block to the stale episode instead of the live thread. The injection path had not received the Phase 1.5 echo defenses; the provenance tool had. The general lesson: in a self-indexing system, *every* retrieval surface — benchmark, analysis tooling, and context injection alike — needs echo defense and margin gating, because each one is a place where the system's record of being asked can outrank the thing that was asked about.

6.3 Limitations

(a) Scale and design. Evaluation uses 12 hand-written queries on a single system and single corpus — no external dataset, no multi-organization coding history. M3 has been upgraded from single-annotator to a blind three-judge cross-vendor panel with moderate agreement (Fleiss $\kappa = 0.51$), but the judges are LLMs rather than humans, and 12 queries remains a small suite.

(b) GT and graph plumbing. GT is built from `code_evolution`, which Arm B also uses in step 3. Circularity is mitigated by seed dependence and by blend-channel dominance in attribution, but not eliminated. An attempted independent GT from VCS commit trailers failed on sparse trailer coverage and on self-contamination of the ID bridge (see Method); the mitigation path is specified but not yet realized.

(c) Observer effect partially mitigated, not solved. Frozen snapshots and Phase 1.5 echo defenses repair the measured displacement cases but do not rescue origin conversations for decisions whose re-askings dominate entire conversations; conversation-level exclusion and `supersedes` chains remain open work.

(d) Episode-chain value revised, not settled. The episode hop contributed zero GT hits in the coverage-proxy era (sparse/young `prev_episode_id` chains); on graded gold the episode-only ablation arm is a strong origin-finder (0.321 origin-MRR) but weak on evidence depth. Its value is now positive but characterized on 12 mapped queries only.

(e) Generalizability. The primary suite is maximally dogfooded (CSR indexing its own development). Within-operator cross-project replication on a second, unrelated corpus (TypeScript product/marketing history, +47% vs. the primary +53%) removes the self-description confound but not the single-operator one: all indexed history is one person's work with one agent stack. Multi-operator and multi-organization validation remain open.

(h) GT layers and their residual weaknesses. The coverage results use a file-touch proxy (all `code_evolution` sessions touching the target file count equally), which counts design-originating, implementing, and merely-discussing sessions as equally correct and shares plumbing with Arm B's graph channel; those results should be read as coverage of *decision-relevant activity*. The graded gold (E2) removes the proxy but has its own limits: grades derive from LLM-extracted dialog-acts (dual-vendor consensus, directs $\kappa = 0.41$ — moderate; splits resolved conservatively and routed to owner audit, which the owner completed only for the five contested origin maps), the gold is same-corpus (mitigated by feature disjointness: grades come from dialog-acts and ship ledgers, retrieval from embeddings and graph structure), and origin-MRR rests on n=12 single-operator sealed recollections.

(f) Concurrent-work race. ACT-Up, MRMS, and E-mem have now been close-read in full (Related Work reflects their actual mechanisms and evaluations), but the space is moving quickly; other concurrent preprints may exist that we have not surveyed, and the composition could be assembled by others within months.

(g) Incomplete three-source modeling. CMR motivates source-factored reinstatement of intent vs. deliberation vs. artifact (Polyn et al., 2009). Phase 1 labels sidechains and stores sequence but still uses a single fused context blend and does not yet weight by `is_sidechain` or `seq`.

6.4 Deferred work and theoretical next steps

The three experiments queued in earlier drafts — the factored ablation grid, graded provenance gold, and the controlled contamination experiment — have been executed and are reported above (Phase 2 experiments). What they leave open sets the queue.

From E1: fusion re-weighting (a larger graph share, per-channel quotas) and a blend-free graph+episode arm. From E2: the origin-finding floor — 5 of 12 owner-verified origins missed by every arm, concentrated in the corpus's earliest era — motivating age-compensating retrieval; and a cross-persona replication with marketing-operations queries against the machine's purpose-built release-train ledger. From E3: conversation-level echo exclusion (chunk-level demotion repairs occupancy but cannot rescue origins whose re-askings dominate whole conversations) and `supersedes` population.

Learned decay (Anderson & Schooler, 1991) is theoretically motivated and now *unblocked* by MCP retrieval logging, but blocked on log accumulation. Chunk-level temporal reinstatement using `seq`, sidechain-aware scoring, and SessionStart saga narration remain non-goals of this phase. External and multi-operator validation is the immediate scientific next step.

7 Conclusion

We introduced *three-trace sagas* — joint episodic structure over intent, deliberation, and artifact in agentic software construction — and *reinstatement recall*, a five-step, LLM-free retrieval algorithm that seeds with kNN, blends query and seed context (TCM analogue), spreads over a session–file code graph, hops episode chains, and fuses results under a provenance-aware rerank. On CSR's own development corpus, a pre-registered spike showed +53% summed GT-session coverage at $k = 10$ versus one-shot kNN, with mechanism attribution locating most gains in semantic context blend rather than graph walk. Production reimplementation as `csr_why` met latency and integration DoD under partial same-day self-contamination ($15 \rightarrow 18/19$), and a frozen-corpus rerank pass repaired the contamination-displaced acceptance query ($0 \rightarrow 1$) while holding aggregate coverage.

Supported on these corpora: reinstatement beats flat kNN for multi-hop provenance questions, at sub-100 ms warm cost, without an LLM in the loop — supported by GT coverage on two unrelated within-operator corpora (+53% Rust/systems, +47% TypeScript/product; pooled sign test $p \approx 0.001$), by blind cross-vendor judge panels on both (8/12 preferred, $\kappa = 0.51$; 5/8 preferred with zero baseline votes), and by ratification-derived graded gold (+37% origin-MRR, +29% `nDCG@10`, +25% graded recall, n=12 owner-sealed origins). Channel ablation attributes the gain to code-graph spread and shows the echo defense is causally necessary (removing it falls below the kNN baseline); controlled contamination dosing shows five re-ask cycles capture half of naive retrieval's top-10 and that the same defense repairs 72% of echo occupancy at 3.4× the origin-MRR, with a null sham control.

Not yet proven: learned usage-weighted decay; sidechain-weighted scoring; chunk-sequence temporal reinstatement; generalization beyond a single operator's history; and origin recovery itself — the verified origin conversation stays outside every arm's top-10 on over half the graded queries, and re-askings that dominate whole conversations remain unrescued by chunk-level defenses.

Immediate next steps: conversation-level echo exclusion and `supersedes` population; enforce session-zero frozen snapshots for all future evals; accumulate MCP retrieval logs for Phase 2 learned decay; validate on external agentic coding corpora.

For agentic construction, Naur's theory is no longer only in human heads — it is stranded across sagas. Systems that index only similar text will keep losing it. Systems that reinstate the encoding context of the work that produced the code have a chance to keep it.

Data availability and privacy

The evaluation corpus is one operator's private development history — session transcripts, subagent traces, and code evolution across personal and commercial projects — and cannot be released as-is. Transcript excerpts quoted in this paper were reviewed by the corpus owner before inclusion. The evaluation *harness* (frozen-snapshot protocol, MCP probe scripts, blinded judging packets, agreement computation) contains no corpus content and is releasable; external replication therefore requires running the harness on one's own agent history, which is exactly the deployment condition the system targets.

References

Anderson, J. R., & Schooler, L. J. (1991). Reflections of the environment in memory. *Psychological Science, 2*(6), 396–408.

Deng, Y. (2026). Entity-collision: A stratified protocol for attributing retrieval lift in agent memory. *arXiv preprint* arXiv:2605.29630.

Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., & Su, Y. (2025). From RAG to memory: Non-parametric continual learning for large language models. *Proceedings of ICML 2025*. arXiv:2502.14802.

Howard, M. W., & Kahana, M. J. (2002). A distributed representation of temporal context. *Journal of Mathematical Psychology, 46*(3), 269–299.

Ko, A. J., DeLine, R., & Venolia, G. (2007). Information needs in collocated software development teams. *Proceedings of ICSE 2007*, 344–353.

Kunz, W., & Rittel, H. W. J. (1970). *Issues as elements of information systems* (Working Paper 131). Institute of Urban and Regional Development, University of California, Berkeley.

LaToza, T. D., & Myers, B. A. (2010). Hard-to-answer questions about code. *Proceedings of PLATEAU 2010*. ACM.

Li, J., & Shi-Nash, A. (2026). MRMS: A multi-resolution memory substrate for long-lived AI agents. *arXiv preprint* arXiv:2607.04617.

MAGMA. (2026). MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint* arXiv:2601.03236.

Moran, T. P., & Carroll, J. M. (Eds.). (1996). *Design rationale: Concepts, techniques, and use*. Lawrence Erlbaum Associates.

Naur, P. (1985). Programming as theory building. *Microprocessing and Microprogramming, 15*(5), 253–261.

Nygard, M. (2011). Documenting architecture decisions. Cognitect blog, November 15, 2011.

Polyn, S. M., Norman, K. A., & Kahana, M. J. (2009). A context maintenance and retrieval model of organizational processes in free recall. *Psychological Review, 116*(1), 129–156.

Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., & Chalef, D. (2025). Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint* arXiv:2501.13956.

Rocchio, J. J. (1971). Relevance feedback in information retrieval. In G. Salton (Ed.), *The SMART Retrieval System: Experiments in Automatic Document Processing* (pp. 313–323). Prentice-Hall.

Thomson, R., & Lebiere, C. (2026). Rapid prototyping of event-driven contextual memory in the ACT-Up cognitive architecture. *arXiv preprint* arXiv:2606.28045.

Wang, et al. (2026). E-mem: Multi-agent based episodic context reconstruction for LLM agent memory. *Proceedings of ICML 2026*, PMLR 306. arXiv:2601.21714.

Wu, R., Zhang, H., Kim, S., & Cheung, S.-C. (2011). ReLink: Recovering links between bugs and changes. *Proceedings of ESEC/FSE 2011*, 15–25.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *Proceedings of NeurIPS 2025*. arXiv:2502.12110.